1
Amazon EC2 and Compute Services

Amazon Web Services (**AWS**) is a cloud computing platform offered by Amazon. It provides a wide range of cloud-based services, including compute, storage, networks, databases, data analytics, **machine learning** (**ML**), and other functionality that can be used to build scalable and flexible applications. We will start our Amazon cloud learning journey from the AWS compute services—specifically, **Elastic Compute Cloud** (**EC2**), which was one of the most basic and earliest cloud services in the world.

In this chapter, we will cover the following topics:

- **The history of computing**: How the first computer evolved from physical to virtual and led to cloud compute
- **Amazon Global Cloud Infrastructure**: Where all the AWS global cloud services are based
- **Building our first EC2 instances in the Amazon cloud**: Provision EC2 instances in the AWS cloud, step by step
- **Elastic Load Balancers** (**ELBs**) and **Auto Scaling Groups** (**ASGs**): The framework providing EC2 services elastically
- **AWS compute – from EC2 to containers to serverless**: Extend from EC2 to other AWS compute services, including **Elastic Container Service** (**ECS**), **Elastic Kubernetes Service** (**EKS**), and Lambda

By following the discussions in this chapter, you will be able to grasp the basic concepts of cloud computing, AWS EC2, and compute services, and gain hands-on skills in provisioning EC2 and compute services. Practice questions are provided to assess your knowledge level, and further reading links are included at the end of the chapter.

The history of computing

In this section, we will briefly review the computing history of human beings, from the first computer to Amazon EC2, and understand what has happened in the past 70+ years and what led us to the cloud computing era.

The computer

The invention of the computer is one of the biggest milestones in human history. On December 10, 1945, **Electronic Numerical Integrator and Computer** (**ENIAC**) was first put to work for practical purposes at the University of Pennsylvania. It weighed about 30 tons, occupied about 1,800 sq ft, and consumed about 150 kW of electricity.

From 1945 to now, in over 75 years, we human beings have made huge progress in upgrading the computer. From ENIAC to desktop and data center servers, laptops, and iPhones, *Figure 1.1* shows the computer evolution landmarks:

Figure 1.1 – Computer evolution landmarks

Let's take some time to examine a computer—say, a desktop PC. If we remove the cover, we will find that it has the following main *hardware* parts—as shown in *Figure 1.2*:

- **Central processing unit** (**CPU**)
- **Random access memory** (**RAM**)
- **Hard disk** (**HD**)
- **Network interface card** (**NIC**)

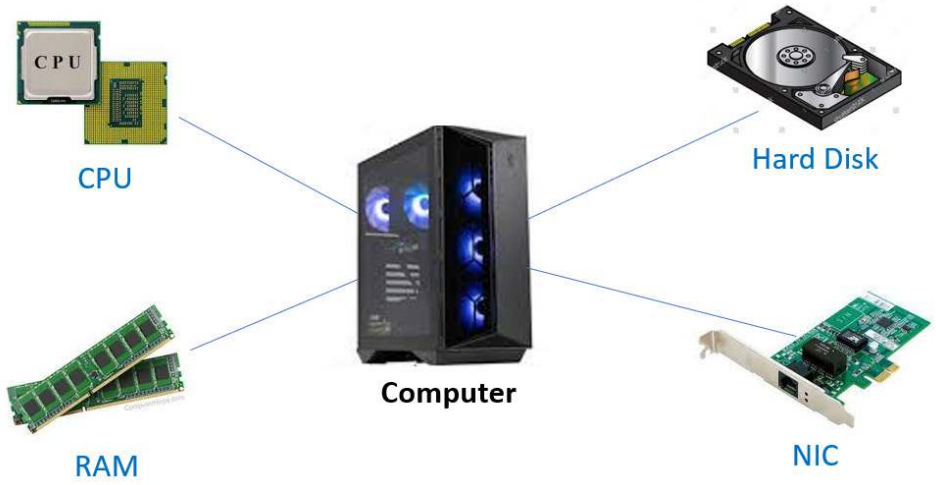

Figure 1.2 – Computer hardware components

These hardware parts work together to make the computer function, along with the *software* including the **operating system** (such as Windows, Linux, macOS, and so on), which manages the hardware, and the **application programs** (such as Microsoft Office, web servers, games, and so on) that run on top of the operating system. In a nutshell, hardware and software specifications decide how much power a computer can serve for different business use cases.

The data center

Apparently, one computer does not serve us well. Computers need to be able to communicate with each other to fulfill network communications, resource sharing, and so on. The work at Stanford University in the 1980s led to the birth of Cisco Systems, Inc., an internet company that played a great part in connecting computers together and forming the intranet and the internet. Connecting many computers together, *data centers* emerged as a central location for computing resources—CPU, RAM, storage, and networking.

Data centers provide resources for businesses' information technology needs: computing, storing, networking, and other services. However, the concept of data center ownership lacks flexibility and agility and entails huge investment and maintenance costs. Often, building a new data center takes a long time and a big amount of money, and maintaining existing data centers—such as tech refresh projects—is very costly. In certain circumstances, it is not even possible to possess the computing resources to complete certain projects. For example, the Human Genome Project was estimated to consume up to 10,000 trillion CPU hours and 40 exabytes (1 exabyte = 10^{18} bytes) of disk storage, and it is impossible to acquire resources at this scale without leveraging cloud computing.

The virtual machine

The peace of physical computers was broken in 1998 when VMware was founded and the concept of a **virtual machine** (**VM**) was brought to Earth. A VM is a software-based computer composed of virtualized components of a physical computer—CPU, RAM, HD, network, operating system, and application programs.

VMware's hypervisor virtualizes hardware to run multiple VMs on bare-metal hardware, and these VMs can run various operating systems of Windows, Linux, or others. With virtualization, a VM is represented by a bunch of files. It can be exported to a binary image that can be deployed on any physical hardware at different locations. A running VM can be *moved* from one host to another, *LIVE*—so-called" v-Motion". The virtualization technologies virtualized physical hardware and caused a revolution in computer history, and also made cloud computing feasible.

The idea of cloud computing

The limitation of data centers and virtualization technology made people explore more flexible and inexpensive ways of using computing resources. The idea of cloud computing started from the concept of "*rental*"—use as needed and pay as you go. It is the on-demand, self-provisioning of computing resources (hardware, software, and so on) that allows you to pay only for what you use. The key concept of cloud computing is *disposable computing resources*. In the traditional information technology and data center concept, a computer (or any other compute resource) is treated as a *pet*. When a pet dies, people are very sad, and they need to get a new replacement right away. If an investment bank's trading server goes down at night, it is the end of the world—everyone is woken up to recover the server. However, in the new cloud computing concept, a computer is treated as cattle in a herd. For example, the website of an investment bank, `zhebank.com`, is supported by a *herd* of 88 servers—`www001` to `www88`. When one server goes down, it's taken out of the serving line, shot, and replaced with another new one with the same configuration and functionalities, automatically!

With cloud computing, enterprises are leveraging the **cloud service provider** (**CSP**)'s *unlimited* computing resources that are featured as global, elastic and scalable, highly reliable and available, cost-effective, and secure. The main CSPs, such as Amazon, Microsoft, and Google, have global data centers that are connected by backbone networks. Because of cloud computing's pay-as-you-go characteristics, it makes sense for cost savings. Because of its strong monitoring and logging features, cloud computing offers the most secure hosting environment. Instead of building physical hardware data centers with big investments over a long time, virtual software-based data centers can be built within several hours, immutable and repeatedly, in the global cloud environment. Infrastructure is represented as code that can be managed with version control, which we can call **Infrastructure as Code** (**IaC**). More details can be found at `https://docs.aws.amazon.com/whitepapers/latest/introduction-devops-aws/infrastructure-as-code.html`.

EC2 was first introduced in 2006 as a web service that allowed customers to rent virtual computers for computing tasks. Since then, it has become one of the most popular cloud computing platforms available, offering a wide range of services and features that make it an attractive option for enterprise

customers. Amazon categorizes the VMs with different EC2 instance types based on hardware (CPU, RAM, HD, and network) and software (operating system and applications) configurations. For different business use cases, cloud consumers can choose EC2 instances with a variety of instance types, operating system choices, network options, storage options, and more. In 2013, Amazon introduced the **Reserved Instance** feature, which gave customers the opportunity to purchase instances at discounted rates in exchange for committing to longer usage terms. In 2017, Amazon released EC2 Fleet, which allows customers to manage multiple instance types and instance sizes across multiple **Availability Zones** (**AZs**) with a single request.

The computer evolution path

From ENIAC to EC2, a computer has evolved from a huge, physical unit to a disposable resource that is flexible and on-demand, portable, and replaceable, and a data center has evolved from being expensive and protracted to a piece of code that can be executed globally at any time on demand, within hours.

In the next sections of this chapter, we will look at the Amazon Global Cloud Infrastructure and then provision our EC2 instances in the cloud.

Amazon Global Cloud infrastructure

The **Amazon Global Cloud Infrastructure** is a suite of cloud computing services offered by AWS, including compute, storage, databases, analytics, networking, mobile, developer tools, management tools, security, identity compliance, and so on. These services are hosted globally, allowing customers to store data and access resources in locations that best meet their business needs. It delivers highly secure, low-cost, and reliable services that can be used by almost any application in any industry around the world.

Amazon has built physical data centers around the world, in graphical areas called AWS Region*s*, which are connected by Amazon's backbone network infrastructure. Each Region provides full redundancy and connectivity among its data centers. An AWS Region typically consists of two or more AZs, which is a fully isolated partition of the AWS infrastructure. An AZ has one or more data centers connected with each other and is identified by a name that combines a letter identifier with the region's name. For example, `us-east-1d` is the *d AZ* in the `us-east-1` region. Each AZ is designed for fault isolation and is connected to other AZs using high-speed private networking. When provisioning cloud resources such as EC2, you choose the region and AZs where the EC2 instance will be sitting. In the next section, we will demonstrate the EC2 instance provisioning process.

Building our first EC2 instances in the Amazon cloud

In this section, we will use the AWS cloud console and CloudShell command line to provision EC2 instances running in the Amazon cloud—Linux and Windows VMs, step by step. Note that the user interface may change, but the procedures are similar.

Before we can launch an EC2 instance, we need to create an AWS account first. Amazon offers a *free tier* account for new cloud learners to provision some basic cloud resources, but you will need a credit card to sign up for an AWS account. Since your credit card is involved, there are three things to keep in mind with your AWS 12-digit account, as follows:

- Enable **multi-factor authentication** (**MFA**) to protect your account
- You can log in to the console with your email address, but be aware that this is the root user, which has the superpower to provision any resources globally
- Clean up all/any cloud resources you have provisioned after completing the labs

Having signed up for an AWS account, you are ready to move to the next phase—launching EC2 instances using the cloud console or CloudShell.

Launching EC2 instances in the AWS cloud console

Logging in to the AWS console at `console.aws.amazon.com`, you can search for EC2 services and launch an EC2 instance by taking the following nine steps:

1. **Select the software of the EC2 instance**: Think of it just like selecting software (OS and other applications) when purchasing a physical desktop or laptop PC.

   In AWS, the software image for an EC2 instance is called an **Amazon Machine Image** (**AMI**), which is a template that is used to launch an EC2 instance. Amazon provides AMIs in Windows, Linux, and other operating systems, customized with some other software pre-installed:

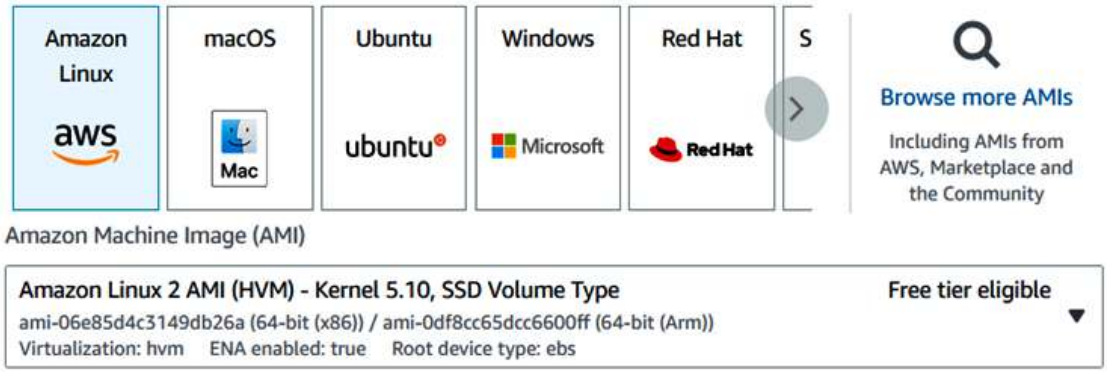

Figure 1.3 – Selecting an AMI

   As shown in *Figure 1.3*, we have chosen the Amazon Linux 2 AMI, which is a customized Linux OS tuned for optimal performance on AWS and easy integration with AWS services, and it is free-tier eligible.

In many enterprises, AMI images are standardized to be used as **seeds** to deploy EC2 instances—we call them **golden images**. A production AMI includes all the packages, patches, and applications that are needed to deploy EC2 instances in production and will be managed with secure version-control management systems.

2. **Select the hardware configuration of the EC2 instance**: This is just like selecting hardware—the number of CPUs, RAM, and HD sizes when purchasing a physical desktop or laptop PC. In AWS, the hardware selection is to choose the right EC2 instance type—Amazon has categorized the EC2 hardware configurations into various instance types, such as **General Purpose**, **Compute Optimized**, **Memory Optimized**, and so on, based on business use cases. Some AWS EC2 instance types are shown in *Figure 1.4*:

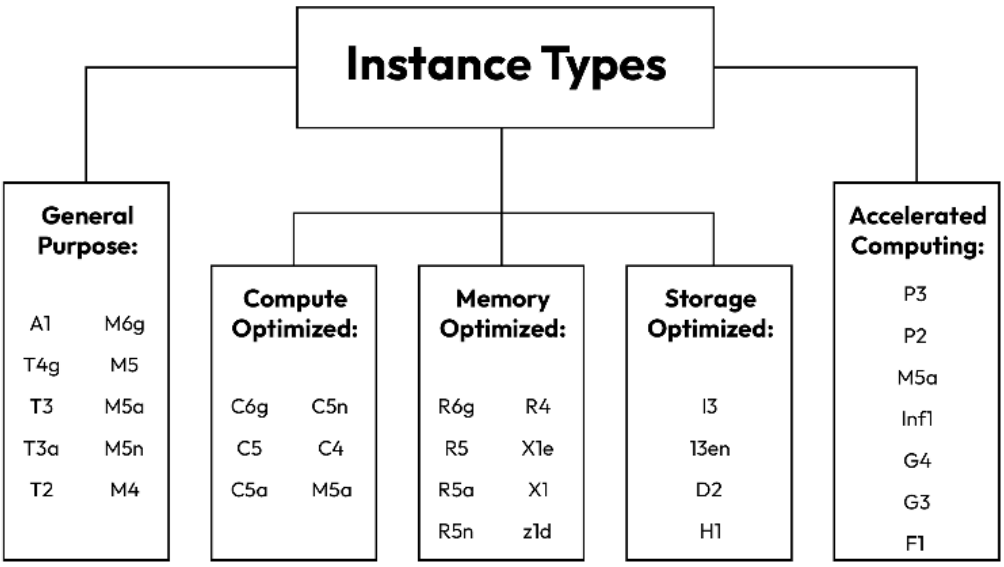

Figure 1.4 – EC2 instance types

Each instance type is specified by a category, family series, generation number, and configuration size. For example, the `p2.8xlarge` instance type can be used for an Accelerated Computing use case, where `p` is the instance family series, `2` is the instance generation, and `8xlarge` indicates its size is 8 times the `p2.large` instance type.

We will choose `t2.micro`, which is inexpensive and free-tier eligible, for our EC2 instances.

3. **Specify the EC2 instance's network settings**: This is like subscribing to an **Internet Service Provider** (**ISP**) for our home PC to connect to a network and the internet. In the AWS cloud, the basic network unit is called a **Virtual Private Cloud** (**VPC**), and Amazon has provided a default VPC and subnets in each region. At this time, we will take the default setting—our first EC2 instance will be placed into the default VPC/subnet and be assigned a public IP address to make it internet-accessible.

4. **Optionally attach an AWS Identity and Access Management (IAM) role to the EC2 instance**: This is something very different from traditional concepts but is very useful for software/applications running on the EC2 instance to interact with other AWS services.

   With IAM, you can specify who can access which resources with what permissions. An IAM role can be created and assigned with permissions to access other AWS resources, such as reading an **Amazon Simple Storage Service** (**Amazon S3**) bucket. By attaching the IAM role to an EC2 instance, all applications running on the EC2 instance will have the same permissions as that role. For example, we can create an IAM role, assign it read/write access to an S3 bucket, and attach the role to an EC2 instance, then all the applications running on the EC2 instance will have read/write access to the S3 bucket. *Figure 1.5* shows the concept of attaching an IAM role to an EC2 instance:

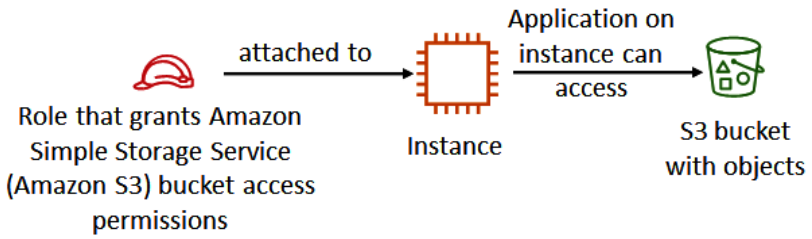

Figure 1.5 – Attaching an IAM role to an EC2 instance

5. **Optionally specify a user data script to the EC2 instance**: User data scripts can be used to customize the runtime environment of the EC2 instance—it executes the first time the instance starts. I have had experience using the EC2 user data script—at a time when the Linux system admin left my company and no one in the company was able to access a Linux instance sitting in the AWS cloud. While there exist many ways to rescue this situation, one interesting solution we used was to generate a new key pair (public key and private key), stop the instance, and leverage the instance's user data script to append the new public key to the `EC2-user` user's **Secure Shell** (**SSH**) profile, during the instance starting process. With the new public key added to the EC2 instance, the `ec2-user` user can SSH into the instance with the new private key.
6. **Optionally attach additional storage volumes to the EC2 instance**: This can be thought of as buying and adding additional disk drives to our PC at home. For each volume, we need to specify the size of the disk (in GB) and the volume type (hardware types), and whether encryption should be used for the volume.
7. **Optionally assign a tag to the EC2 instance**: A tag is a label that we can assign to an AWS resource, and it consists of a key and an optional value. With tags, we attach metadata to cloud resources such as an EC2 instance. There are many potential benefits of tagging in managing cloud resources, such as filtering, automation, cost allocation and chargeback, and access control.

8. **Setting a Security Group (SG) for the EC2 instance**: Just like configuring firewalls on our home routers to manage access to our home PCs, an SG is a set of firewall rules that control traffic to and from our EC2 instance. With an SG, we can create rules that specify the source (for example, an IP address or another SG), the port number, and the protocol, such as HTTP/HTTPS, SSH (port `22`), or **Internet Control Message Protocol** (**ICMP**). For example, if we use the EC2 instance to host a web server, then the SG will need an SG rule to open ports for `http` (`80`) and `https` (`443`). Note that SGs exist outside of the instance's guest OS—traffic to the instance can be controlled by both SGs and guest OS firewall settings.
9. **Specify an existing key pair or create a new key pair for the EC2 instance**: A key pair consists of a public key that AWS stores on the instance and a private key file that you download and store on your local computer for remote access. When you try to connect to the instance, the keys from both ends are matched to authenticate the remote user/connections. For Windows instances, we need to decrypt the key pair to obtain the administrator password for logging in to the EC2 instance remotely. For Linux instances, we utilize the private key and use SSH to securely connect to the cloud instance. Note that the only chance to download an EC2 key pair is during the instance creation time. If you've lost the key pair, you cannot recover it. The only workaround is to create an AMI of the existing instance, and then launch a new instance with the AMI and a new key pair. Also, note that there are two formats for an EC2 key pair when you save it to the local computer: the `.pem` format is used on Linux-based terminals including Mac, and the `.ppk` format is used for Windows.

Following the preceding nine steps, we have provisioned our first EC2 instance—a Linux VM in the AWS cloud. Following the same procedure, let us launch a Windows VM. The only difference is that in *step 1*, we choose the Microsoft Windows operating system— specifically, **Microsoft Windows Server 2022 Base**—as shown in *Figure 1.6*, which is also free-tier eligible:

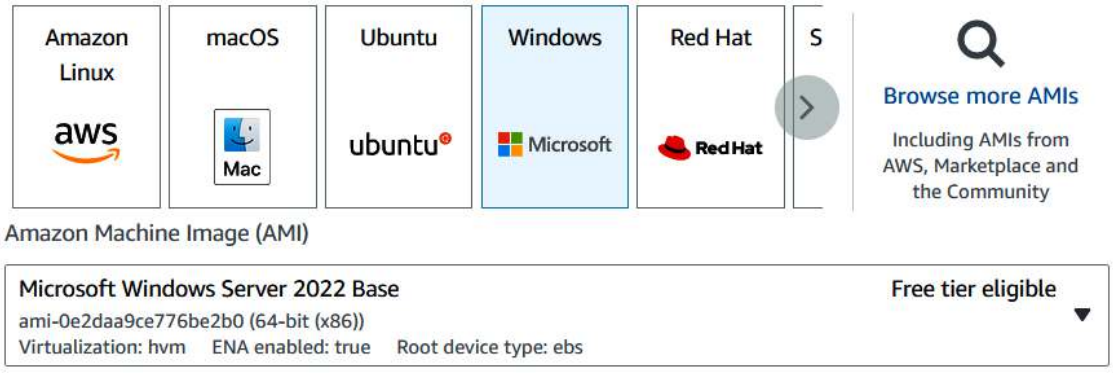

Figure 1.6 – Selecting Microsoft Windows as the operating system

So far, we have created two EC2 instances in our AWS cloud—one Linux VM and one Windows VM—via the AWS Management console.

Launching EC2 instances using CloudShell

We can also launch EC2 instances using the command line in CloudShell, which is a browser-based, pre-authenticated shell that you can launch directly from the AWS Management Console. Next are detailed steps to create an EC2 Windows instance in the `us-west-2` Region:

1. **From the AWS console, launch CloudShell** by clicking the CloudShell sign, as shown in *Figure 1.7*:

Figure 1.7 – Launching CloudShell from the AWS console

2. **Find the AWS AMI image ID** in the `us-west-2` region, with the following CloudShell command – the results are shown in *Figure 1.8*:

```
[cloudshell-user]$ aws ec2 describe-images --region us-west-2
```

Figure 1.8 – Finding the Linux AMI image ID

3. **Find the SG name** we created in the previous section, as shown in *Figure 1.9*:

```
[cloudshell-user@ip-10-6-69-82 ~]$ aws ec2 describe-security-groups
{
    "SecurityGroups": [
        {
            "Description": "launch-wizard-1 created 2023-01-26T22:34:24.105Z",
            "GroupName": "launch-wizard-1",
            "IpPermissions": [
                {
                    "FromPort": 3389,
                    "IpProtocol": "tcp",
                    "IpRanges": [
                        {
                            "CidrIp": "129.110.242.32/32"
                        }
```

Figure 1.9 – Finding the SG name

4. **Find the key pair** we created in the previous section, as shown in *Figure 1.10*:

```
[cloudshell-user@ip-10-6-69-82 ~]$ aws ec2 describe-key-pairs
{
    "KeyPairs": [
        {
            "KeyPairId": "key-092be0798f7c7883b",
            "KeyFingerprint": "c7:4d:81:09:8c:3d:cb:db:b0:47:ee:4e:21:dd:53:cd:54:5a:65:72",
            "KeyName": "mywestkp",
            "KeyType": "rsa",
            "Tags": [],
            "CreateTime": "2023-01-26T22:35:40+00:00"
        }
    ]
}
```

Figure 1.10 – Finding the key pair name

5. **Create an EC2 instance** in the `us-west-2` region, using the `aws ec2 run-instances` command, with the following configurations we obtained from the previous steps. A screenshot is shown in *Figure 1.11*. The instance ID is called out from the output

   AMI:

   `ami-0ef0b498cd3fe129c`

   SG:

   `launch-wizard-1`

   Key pair:

`mywestkp`

Instance type:

`t2.micro`

```
aws ec2 run-instances --image-id ami-0ef0b498cd3fe129c --count 1
--instance-type t2.micro --key-name mywestkp --security-groups
launch-wizard-1 --region us-west-2
```

Figure 1.11 – Launching an EC2 instance

6. **Examine the details of the instance** from its `InstanceId` value. As shown in *Figure 1.12*, the instance has a public IP address of `35.93.143.38`:

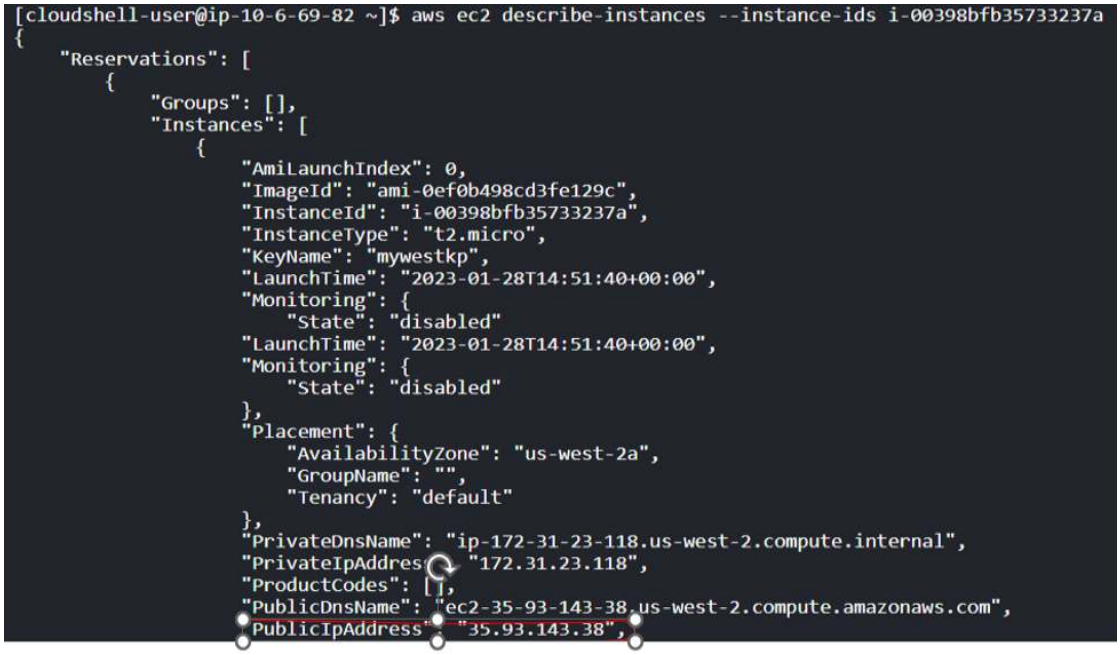

Figure 1.12 – Finding the EC2 instance's public IP address

So far, we have created another EC2 instance using CloudShell with command lines. Note that CloudShell allows us to provision any cloud resources using lines of code, and we will provide more examples in the rest of the book.

Logging in to the EC2 instances

After the instances are created, how do we access them?

SSH is a cryptographic network protocol for operating network services securely over an unsecured network. We can use SSH to access the Linux EC2 instance. **PuTTY** is a free and open source terminal emulator, serial console, and network file transfer application. We will download PuTTY and use it to connect to the Linux VM in the AWS cloud, as shown in *Figure 1.13*:

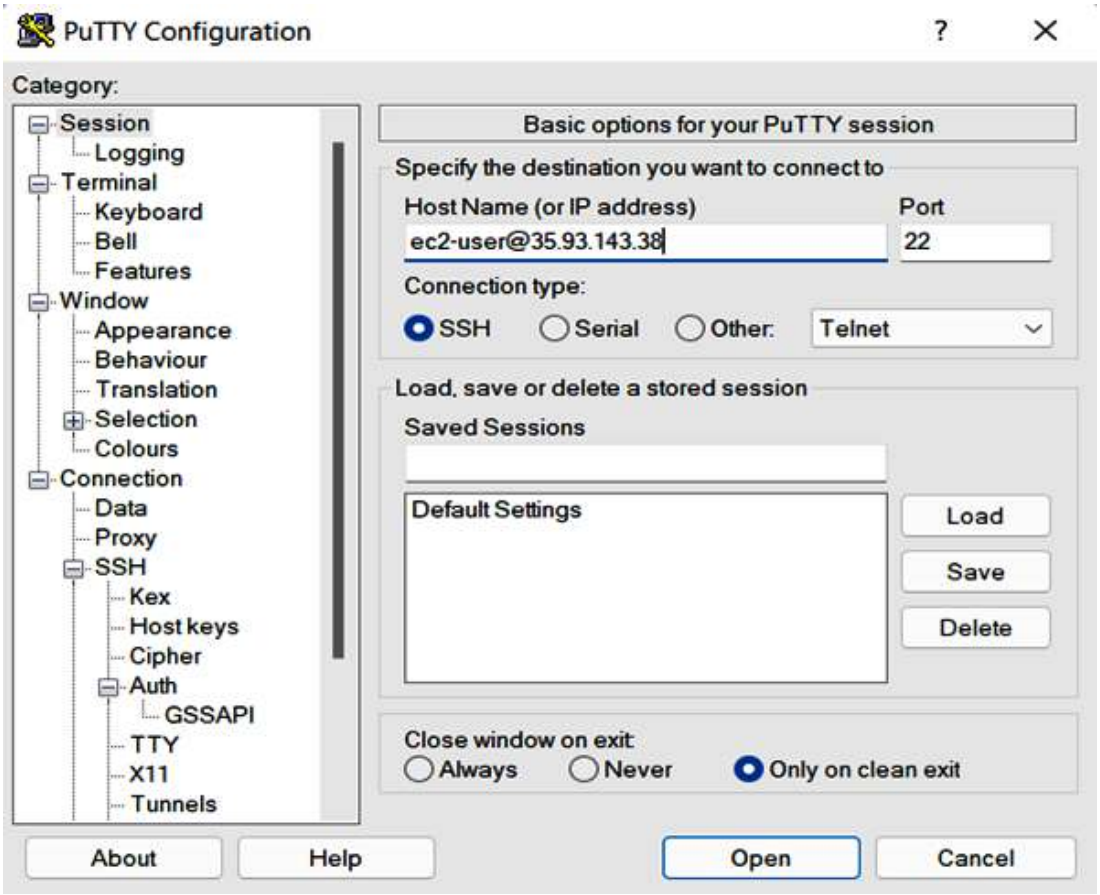

Figure 1.13 – Using PuTTY to connect to the Linux instance

As shown in *Figure 1.13*, we entered `ec2-user@35.93.143.38` in the **Host Name (or IP address)** field. `ec2-user` is a default user created in the guest Linux OS, and `35.93.143.38` is the public IP of the EC2 instance. Note we need to open the SSH port (`22`) in the EC2 instance's SG to allow traffic from our remote machine, as discussed in *step 8* of the *Launching EC2 instances in the AWS cloud console* section earlier in the chapter.

We also need to provide the key pair in the **PuTTY Configuration** window by going to **Connection | SSH | Auth**, as shown in *Figure 1.14*:

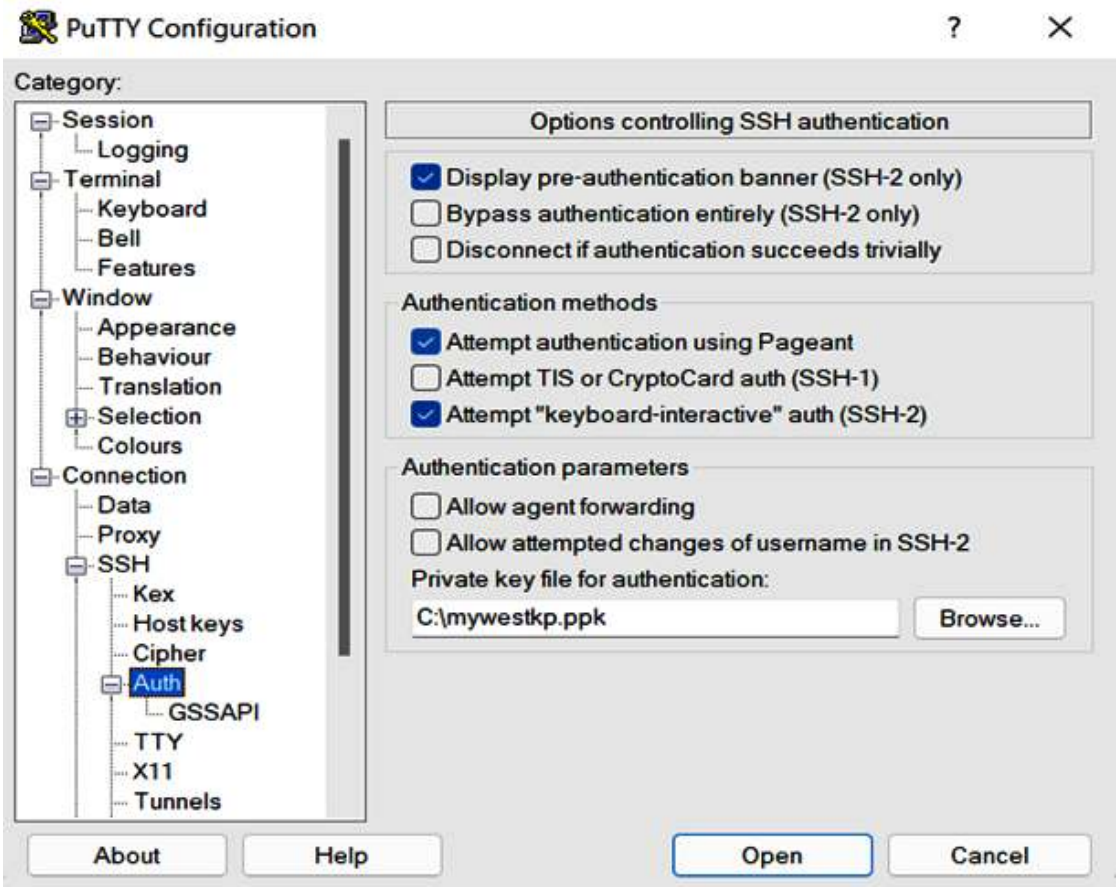

Figure 1.14 – Entering the key pair in PuTTY

Click **Open**, and you will be able to SSH into the Linux instance now. As shown in *Figure 1.15*, we have SSH-ed into the cloud EC2 instance:

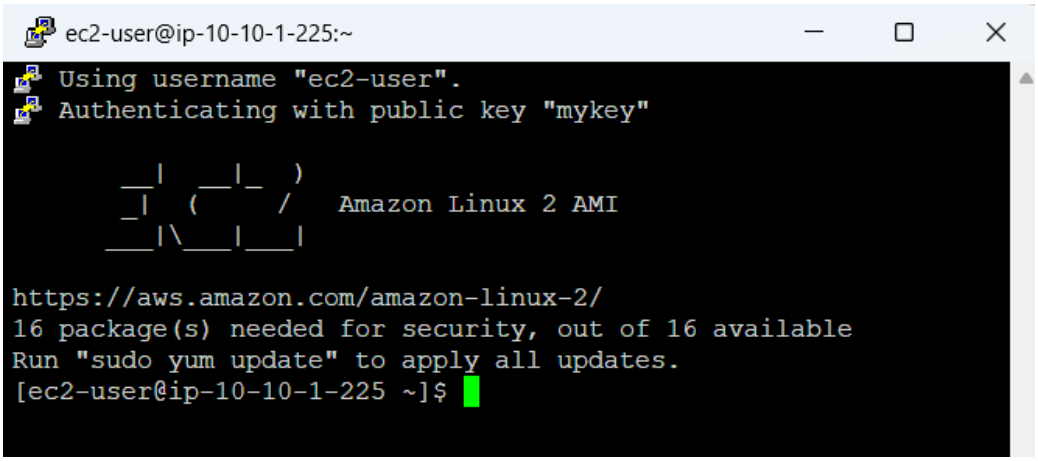

Figure 1.15 – SSH-ing into ec2-1 from the internet

Since we are using a Windows terminal to connect to the remote Linux instance, the key pair format is `.ppk`. If you are using a Mac or another Linux-based terminal, you will need to use the `.pem` format. These two formats can be converted using the open source software PuTTYgen, which is part of the PuTTY family.

With a Linux-based terminal including Mac, use the following command to connect to the cloud Linux EC2 instance:

```
ssh -i keypair.pem ec2-user@35.93.143.38
```

`keypair.pem` is the key pair file in `.pem` format. Make sure it's set to the right permission using the `chomd 400 keypair.pem` Linux command. `ec2-user@35.93.143.38` is `user@EC2`'s public IP address. The default user may change to `ubuntu` if the EC2 instance is an Ubuntu Linux distribution.

For the Windows EC2 instance, just as we access another PC at our home using **Remote Desktop Protocol** (**RDP**), a proprietary protocol developed by Microsoft that provides a user with a graphical interface to connect to another computer over a network connection, we use RDP to log in to the Windows EC2 instance in the AWS cloud. By default, RDP client software is installed on our desktop or laptop, and the Windows EC2 instance has RDP server software running, so it becomes very handy to connect our desktop/laptop to the Windows VM in the cloud. One extra step is that we need to decrypt the administrator's password from the key pair we downloaded during the instance launching process, by going to the AWS console's **EC2 dashboard** and clicking **Instance** | **Connect** | **RDP Client**.

ELB and ASG

We previously briefed the "cattle in a herd" analogy in cloud computing. In this section, we will explain the actual implementation using ELBs and ASGs and use an example to illustrate the mechanism.

An ELB automatically distributes the incoming traffic (workload) across multiple targets, such as EC2 instances, in one or more AZs, so as to balance the workload for high performance and **high availability** (**HA**). An ELB monitors the health of its registered targets and distributes traffic only to the healthy targets.

Behind an ELB, there is usually an ASG that manages the fleet of ELB targets—EC2 instances, in our case. ASG monitors the workload of the instances and uses auto-scaling policies to scale—when the workload reaches a certain up-threshold, such as CPU utilization of 80%, ASG will launch new EC2s and add them into the fleet to offload the traffic until the utilization drops below the up-threshold. When the workload reaches a certain down-threshold, such as CPU utilization of 30%, ASG will shut down EC2s from the fleet until the utilization rises above the threshold. ASG also utilizes a health-check to monitor the instances and replace unhealthy ones as needed. During the auto-scaling process, ASG makes sure that the running EC2 instances are loaded within the thresholds and are laid out across as many AZs in a region.

Let us illustrate ELB and ASG with an example. `www.zbestbuy.com` is an international online e-commerce retailer. During normal business hours, it needs a certain number of web servers to work together to meet online shopping traffic. To meet the global traffic requirements, three web servers are built in different AWS regions—North Virginia (`us-east-1`), London (`eu-west-2`), and Singapore (`ap-southeast-1`). Depending on the customer browser location, Amazon Route 53 (an AWS DNS service) will route the traffic to the nearest web server: when customers in Europe browse the retailer website, the traffic will be routed to the `eu-west-2` web server, which is really an ELB (or **Application Load Balancer** (**ALB**)), and distributed to the EC2 instances behind the ELB, as shown in *Figure 1.16*.

When Black Friday comes, the traffic increases and hits the ELB, which passes the traffic to the EC2 instance fleet. The heavy traffic will raise the EC2 instances' CPU utilization to reach the up-threshold of 80%. Based on the auto-scaling policy, an alarm will be kicked off and the ASG will automatically scale, launching more EC2 instances to join the EC2 fleet. With more EC2s joining in, the CPU utilization will be dropped. Depending on the Black Friday traffic fluctuation, the ASG will always keep up to make sure enough EC2s are working on the workload with normal CPU utilization. When Black Friday sales end, the traffic decreases and thus causes the instances' CPU utilization to drop. When it reaches the down-threshold of 30%, the ASG will start shutting down EC2s based on the auto-scaling policy:

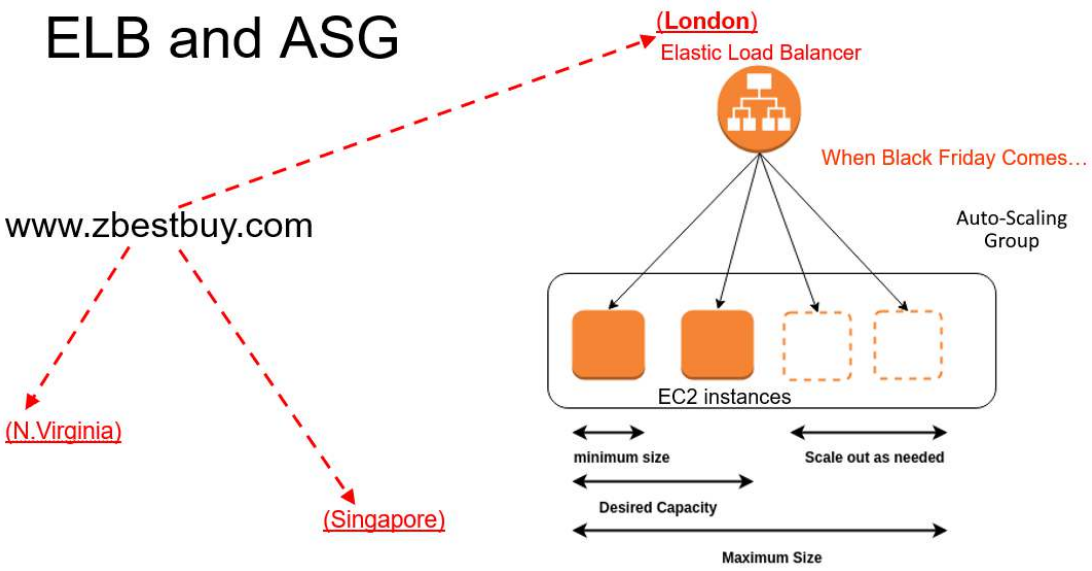

Figure 1.16 – ELB and ASG

As we can see from the preceding example, ELB and ASG work together to scale elastically. Please refer to `https://docs.aws.amazon.com/autoscaling/ec2/userguide/autoscaling-load-balancer.html` for more details.

AWS compute – from EC2 to containers to serverless

So far in this chapter, we have dived into the AWS EC2 service and discussed AWS ELB and ASG. Now, let's spend some time expanding to the other AWS compute services: ECS, EKS, and Lambda (serverless service).

We have discussed the virtualization technology led by VMware at the beginning of the 21st century. While transforming from physical machines to VMs is a great milestone, there still exist constraints from the application point of view: every time we need to deploy an application, we need to run a VM first. The application is also tied up with the OS platform and lacks flexibility and portability. To solve such problems, the concept of **Docker** and **containers** came into the world. A Docker engine virtualizes an OS to multiple apps/containers. A Docker image is a lightweight, standalone, executable package of software that includes everything needed to run an application: code, runtime, system tools, system libraries, and settings. A container is a runtime of the Docker image, and the application runs quickly and reliably from one computing environment to another. Multiple containers can run on the same VM and share the OS kernel with other containers, each running as isolated processes in user space. To further achieve fast and robust deployments and low lead times, the concept of **serverless** computing emerged. With serverless computing, workloads run on servers behind the scenes. From a developer or user's point of view, what they need to do is just submit the code and get the running results back—there is no hassle of building and managing any infrastructure platforms at all, while

resources can continuously scale and be dynamically allocated as needed, yet you never pay for idle time as it is pay per usage.

From VM to container to serverless, Amazon provides EC2, ECS/EKS, and Lambda services correspondingly.

Amazon ECS is a fully managed container orchestration service that helps you easily deploy, manage, and scale containerized applications using Docker or Kubernetes. Amazon ECS provides a highly available and scalable platform for running container-based applications. Enterprises use ECS to grow and manage enterprise application portfolios, scale web applications, perform batch processing, and run services to deliver better services to users.

Amazon EKS, on the other hand, is a fully managed service that makes it easy to deploy, manage, and scale Kubernetes in the AWS cloud. Amazon EKS leverages the global cloud's performance, scale, reliability, and availability, and integrates it with other AWS services such as networking, storage, and security services.

Amazon Lambda was introduced in November 2014. It is an event-driven, serverless computing service that runs code in response to events and automatically manages the computing resources required by that code. Amazon Lambda provides HA with automatic scaling, cost optimization, and security. It supports multiple programming languages, environment variables, and tight integration with other AWS services.

For more details about the aforementioned AWS services and their implementations, please refer to the *Further reading* section at the end of the chapter.

Summary

Congratulations! We have completed the first chapter of our AWS self-learning journey: cloud compute services. In this chapter, we have thoroughly discussed Amazon EC2 instances and provisioned EC2 instances step by step, using the AWS cloud console and CloudShell command lines. We then extended from EC2 (VM) to the container and serverless concepts and briefly discussed Amazon's ECS, EKS, and Lambda services.

In the next chapter, we will discuss Amazon *storage* services, including block storage and network storage that can be added and shared by EC2 instances, and the Simple Storage Service.

At the end of each chapter, we provide practice questions and answers. These questions are designed to help you understand the cloud concepts discussed in the chapter. Please spend time on each question before checking the answer.

Practice questions

1. Which of the following is not a valid source option when configuring SG rules for an EC2 instance?

    A. Tag name for another EC2 instance

    B. IP address for another EC2 instance

    C. IP address ranges for a network

    D. SG name used by another EC2 instance

2. An AWS cloud engineer signed up for a new AWS account, then logged in to the account and created a Linux EC2 instance in the default VPC/subnet. They were able to SSH to the EC2 instance. From the EC2 instance, They:

    A. can access `www.google.com`

    B. cannot access `www.google.com`

    C. can access `www.google.com` only after they configure SG rules

    D. can access `www.google.com` only after they configure **Network Access Control List** (**NACL**) rules

3. Alice launched an EC2 Linux instance in the AWS cloud, and then successfully SSH-ed to the instance from her laptop at home with the default `ec2-user` username. Which keys are used during this process?

    A. `ec2-user`'s public key, which is stored on the EC2 instance, and the private key on the laptop

    B. The `root` user's public key on the EC2 instance

    C. `ec2-user`'s public key, which is stored on the laptop

    D. `ec2-user`'s private key, which is stored on the cloud EC2 instance

    E. `ec2-user`'s symmetric key, which is stored on both the laptop and EC2 instance

4. `www.zbestbuy.com` is configured with ELB and ASG. At peak time, it needs 10 AWS EC2 instances. How do you make sure the website will never be down and can scale as needed?

    A. Set ASG's `minimum instances = 2, maximum instances = 10`

    B. Set ASG's `minimum instances = 1, maximum instances = 10`

    C. Set ASG's `minimum instances = 0, maximum instances = 10`

    D. Set ASG's `minimum instances = 2, maximum instances = 2`

5. A middle school has an education application system using ASG to automatically scale resources as needed. The students report that every morning at 8:30 A.M., the system becomes very slow for about 15 minutes. Initial checking shows that a large percentage of the classes start at 8:30 A.M., and it does not have enough time to scale out to meet the demand. How can we resolve this problem?

A. Schedule the ASGs accordingly to scale out the necessary resources at 8:15 A.M. every morning

B. Use Reserved Instances to ensure the system has reserved the capacity for scale-up events

C. Change the ASG to scale based on network utilization

D. Permanently keep the running instances that are needed at 8:30 A.M. to guarantee available resources

6. AWS engineer Alice is launching an EC2 instance to host a web server. How should Alice configure the EC2 instance's SG?

A. Open ports `80` and `443` inbound to `0.0.0.0/0`

B. Open ports `80` and `443` outbound to `0.0.0.0/0`

C. Open ports `80` and `443` inbound to `10.10.10.0/24`

D. Open ports `80` and `443` outbound to `my IP`

7. An AWS cloud engineer signed up for a new AWS account, then logged in to the account and created an EC2-1 Windows instance and an EC2-2 Linux instance in one subnet (172.31.48.0/20) in the default VPC, using an SG that has `SSH` and `RDP` open to `172.31.0.0/16` only. They were able to RDP to the EC2-1 instance. From the EC2-1 instance, they:

A. can SSH to EC2-2

B. can ping EC2-2

C. cannot ping EC2-1

D. cannot SSH to EC2-2

8. `www.zbestbuy.com` has a need for 10,000 EC2 instances in the next 3 years. What should they use to get these computing resources?

A. Reserved Instances

B. Spot Instances

C. On-demand instances

D. Dedicated-host instances

9. AWS engineer Alice needs to log in to an EC2-100 Linux instance that no one can access since the AWS engineer who was managing it left the company. What does Alice need to do?

    A. Generate a key pair, and add the public key to EC2-100 using `user-data`

    B. Generate a key pair, and add the public key to EC2-100 using `meta-data`

    C. Generate a key pair, and copy the public key to EC2-100 using **Secure Copy Protocol** (**SCP**)

    D. Remove the old private key from EC2-100

10. An AWS architect launched an EC2 instance using the `t2.large` type, installed databases and web applications on the instance, then found that the instance was too small, so they want to move to an `M4.xlarge` instance type. What do they need to do?

Answers to the practice questions

1. A
2. A
3. A
4. A
5. A
6. A
7. A
8. A
9. A
10. One way is to stop the instance in the AWS console and start it with the `M4.xlarge` instance type.

Further reading

For further insights into what you've learned in this chapter, refer to the following links:

- `https://aws.amazon.com/ec2/`
- `https://aws.amazon.com/ecs/`
- `https://aws.amazon.com/eks/`
- `https://aws.amazon.com/lambda/`

- https://docs.aws.amazon.com/autoscaling/ec2/userguide/autoscaling-load-balancer.html
- https://docs.aws.amazon.com/ec2/
- https://aws.amazon.com/blogs/compute/